Spam Mail Prediction System

Using Machine Learning & Logistic Regression

IBM NASSCOM VIRTUAL INTERNSHIP PROGRAM

Student: Anshika (Batch 4)

Email: anshika.akm@gmail.com

Instructor: Mr. Shubham Pathak

Project Overview

A deep dive into building a robust binary classifier to distinguish legitimate (Ham) mail from unsolicited (Spam) messages using Python.

Problem & Objective

The Problem

Unwanted spam emails clutter inboxes and pose security risks like phishing. Manual filtering is inefficient.

The Objective

Develop an automated system to classify emails as 'Spam' or 'Ham' with high accuracy using ML.

Methodology

Utilizing Natural Language Processing (TF-IDF) and Supervised Learning (Logistic Regression).

Dataset Exploration

Google Colab - Notebook Output

```
# Checking dataset dimensions and
data.info(
)
RangeIndex: 5572 entries, 0 to 5571
 Data columns (total 2 columns):
 # Column   Non-Null   Count   Dtype
 0 Category 5572 non-null object
 1 Message  5572 non-null object
 dtypes: object(2)
```

The dataset consists of **5,572** messages. It is a clean dataset with no missing values, containing two primary columns: 'Category' and 'Message'.

Data Preprocessing & Encoding

Label Encoding Step

```
# Converting labels to numeric values
data.loc[data['Category'] == 'spam', 'Category'] = 0
data.loc[data['Category'] == 'ham', 'Category'] = 1

# Defining Features (X) and Labels
X = data['Message']
Y = data['Category']
```

Preparing for the Model

Machine Learning models require numerical input. Here, we perform **Label Encoding**:

- ✓ **Spam** is mapped to **0**
- ✓ **Ham** is mapped to **1**
- ✓ Data is separated into Features (X) and Target (Y)

Dataset Splitting Strategy

80/20

Train-Test Split

```
fro  sklearn.model_selection impor  train_test_split
m                               t
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0. , random_state=3
)                      2
```

We use 80% of the data (4,457 entries) for **Training** the model and 20% (1,115 entries) for **Testing** to evaluate its unseen performance.

Feature Extraction: TF-IDF

TF-IDF Vectorizer

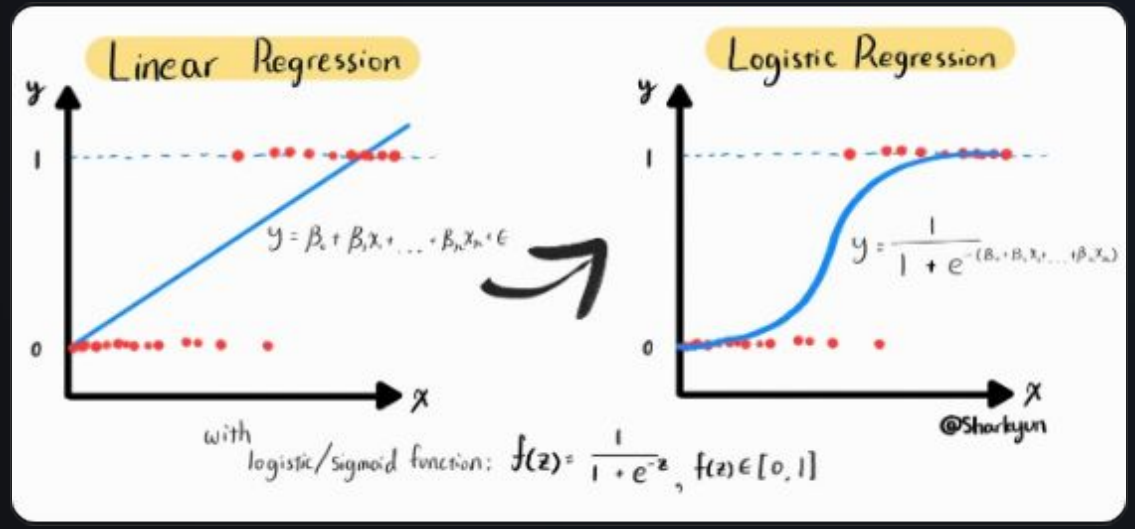
Term Frequency-Inverse Document Frequency converts text into a meaningful representation of numbers based on word importance across documents.

```
feature_extraction = TfidfVectorizer(
  min_df=1,
  stop_words='english',
  lowercase=True
)
X_train_features = feature_extraction.fit_transform(X_train)
X_test_features = feature_extraction.transform(X_test)
```

Model Selection: Logistic Regression

Logistic Regression is a statistical model used for binary classification. It calculates the probability of an entry belonging to a specific class using the Sigmoid function.

$$f(z) = \frac{1}{1 + e^{-z}}$$



Model Evaluation Results

The model demonstrates **excellent generalization**, with nearly identical performance on both seen (training) and unseen (test) data, indicating zero overfitting.

Classification in Action

SPAM

"Win a $1000 gift card now! Click the link below to claim your prize. Limited time offer!"

HAM

"Hey, are we still meeting for lunch at 1 PM today? Let me know if the time works for you."

The Result

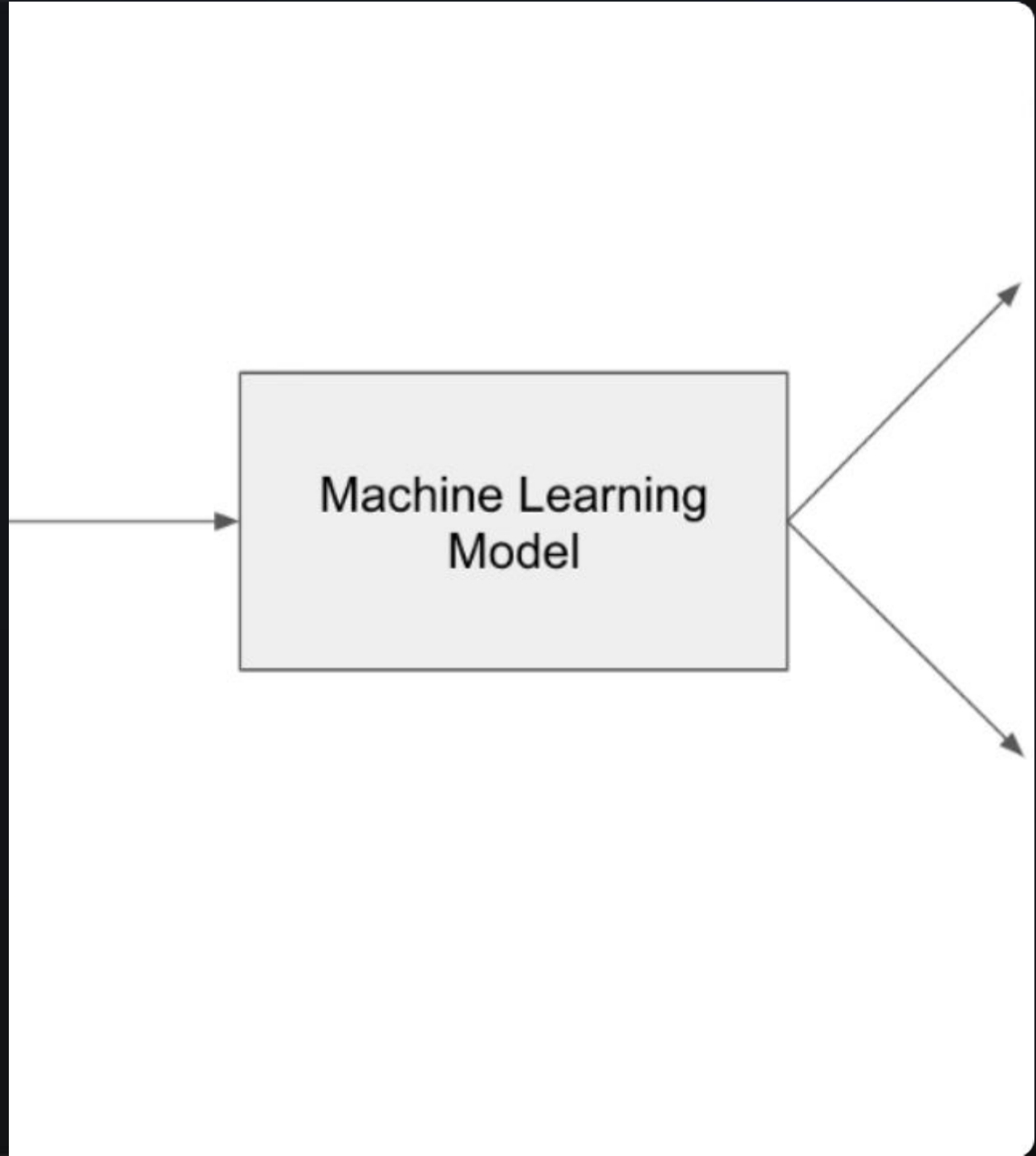
Our system identifies patterns like "Win", "Prize", and "Limited time" to accurately flag Spam.

Real-time Prediction

```
input_mail = ["I've been searching for the right words..." ]
input_features = extraction.transform(input_mail)
prediction = model.predict(input_features)

if prediction[0] == 1:
  print('Ham mail')
else:
  print('Spam mail')
```

The predictive system allows users to input any mail content and get an instant classification based on the trained weights.



Thank You!

Any Questions?

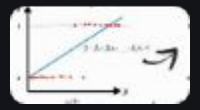
Presented by: **Anshika**

IBM NASSCOM Virtual Internship Program

Image Sources

https://png.pngtree.com/thumb_back/fh260/background/20250319/pngtree-laptop-screen-displaying-lines-of-colorful-code-image_17105612.jpg

Source: pngtree.com



https://miro.medium.com/1*QwUVz0FdpSTzoxh8Y_Sh2A.jpeg

Source: sharkyun.medium.com

https://files.codingninjas.in/article_images/binary-classification-0-1641903394.webp

Source: www.naukri.com